

TUGAS SORTING BINARY TREE



Oleh:

Achmad Mohammad Rivaldi

25091397061

2025B

Mata Kuliah:

Struktur Data

PROGRAM STUDI S1 TERAPAN MANAJEMEN INFORMATIKA

FAKULTAS VOKASI

UNIVERSITAS NEGERI SURABAYA

2026

BAGIAN 1 - Sorting Lanjutan

a. Ruang & Distribusi Sort

Mengapa Radix Sort (array of 10 queues) melanggar $O(1)$?

Radix Sort standar mengalokasikan 10 queue (satu per digit 0–9). Setiap queue dapat menampung hingga n elemen, sehingga total ruang tambahan yang dibutuhkan adalah $O(n)$. Ini jelas melanggar batasan $O(1)$ ekstra pada Modul A.

Bagaimana tmpArray tunggal pada Improved Merge Sort menekan overhead?

Versi berbasis slice (`arr[:mid]`, `arr[mid:]`) membuat salinan fisik subarray baru di setiap level rekursi. Jika ada $\log n$ level rekursi dan setiap level menyimpan total n elemen, overhead ruang akumulatif bisa mencapai $O(n \log n)$.

Sebaliknya, Improved Merge Sort (Listing 12.2 & 12.4) hanya mengalokasikan **satu** tmpArray berukuran n di awal, sekali saja. Semua operasi merge menggunakan virtual sublists — ditentukan oleh indeks (`left_start`, `mid`, `right_end`) — bukan salinan fisik. tmpArray hanya dipakai sebagai buffer sementara saat merge, lalu hasilnya disalin kembali ke `arr` in-place. Total ruang tambahan tetap $O(n)$, dan tidak ada alokasi baru di setiap rekursi.

Strategi modifikasi Radix Sort agar $O(1)$?

Opsi paling realistis adalah **American Flag Sort** (in-place radix sort):

1. Lakukan dua pass per digit: pass pertama hitung frekuensi tiap bucket (hanya butuh array `count[10]` $\rightarrow O(1)$ karena ukuran tetap 10), pass kedua lakukan permutasi in-place menggunakan counting.
2. Karena jumlah bucket selalu 10 (konstan), array `count[10]` dianggap $O(1)$.

Dampak terhadap kompleksitas waktu: tetap $O(d \cdot n)$ secara teori, tetapi konstanta lebih besar karena dua pass per digit dan operasi swap in-place lebih kompleks daripada enqueue/dequeue.

b. Linked List & Pointer Manipulation

Bagaimana `_splitLinkedList()` menemukan titik tengah dalam satu traversal?

Teknik fast-slow pointer (Floyd's tortoise and hare):

- `midPoint` (slow) bergerak 1 langkah per iterasi.
- `curNode` (fast) bergerak 2 langkah per iterasi.

Ketika `curNode` mencapai akhir list (`None` atau node terakhir), `midPoint` tepat berada di tengah. Ini karena rasio kecepatan 2:1, sehingga saat fast menempuh n langkah, slow hanya menempuh $n/2$ langkah. Tidak perlu menghitung panjang list terlebih dahulu — satu traversal cukup.

Setelah loop: `right_head = midPoint.next`, lalu `midPoint.next = None` untuk memutus list menjadi dua bagian.

Mengapa dummy node + tail reference tidak membutuhkan alokasi memori tambahan?

`dummy = ListNode(0)` hanya dibuat **satu kali per panggilan merge** (bukan per node). Selama proses merge, kita hanya **memindahkan pointer `.next`** dari node yang sudah ada — tidak membuat node baru. `tail.next = listA` atau `tail.next = listB` hanya mengubah referensi, bukan mengalokasikan memori.

Kompleksitas ruang:

- Memori heap: $O(1)$ per merge (hanya 1 dummy node).
- Stack rekursi: $O(\log n)$ karena Merge Sort membagi list secara rekursif sebanyak $\log n$ level.
- Total: $O(\log n)$.

c. Quick Sort Worst-Case & Pivot Strategy

Mengapa data descending menyebabkan $O(n^2)$ pada Quick Sort?

Jika pivot selalu dipilih dari elemen pertama (atau terakhir), dan data sudah terurut descending:

- Setiap partisi menghasilkan subarray kiri berukuran 0 dan subarray kanan berukuran $n-1$.
- left bergerak dari kiri menemukan elemen $>$ pivot dengan cepat (hampir tidak bergerak).
- right bergerak dari kanan harus menelusuri seluruh sisa array sebelum berpapasan dengan left.
- Hasilnya: $n-1$ level rekursi, masing-masing melakukan $O(n)$ perbandingan $\rightarrow$ total $O(n^2)$.
- Kedalaman rekursi mencapai $O(n)$, berpotensi stack overflow pada data besar.

Strategi pivot yang lebih robust: Median-of-Three

Pilih median dari $arr[first]$, $arr[mid]$, $arr[last]$ sebagai pivot. Ini memastikan pivot tidak selalu elemen terkecil/terbesar, sehingga partisi lebih seimbang pada data terurut maupun hampir terurut.

Kelayakan pada Singly Linked List:

Median-of-Three membutuhkan akses ke tiga posisi: first, mid, last. Pada singly linked list:

- Akses ke elemen tengah membutuhkan traversal $O(n/2)$.
- Akses ke elemen terakhir membutuhkan traversal $O(n)$.
- Total overhead pemilihan pivot: $O(n)$ per level rekursi.

Ini membuat Quick Sort + Median-of-Three pada linked list tetap $O(n \log n)$ rata-rata, tetapi dengan konstanta lebih besar. Alternatif yang lebih praktis untuk linked list adalah tetap menggunakan **Merge Sort**, karena pemisahan $O(n)$ dengan fast-

slow pointer jauh lebih efisien daripada traversal $O(n)$ berulang untuk pemilihan pivot.

d. Batas Teoretis & Paradigma Sorting

Mengapa Radix Sort $O(dn)$ tidak kontradiktif dengan batas bawah $\Omega(n \log n)$?

Batas bawah $\Omega(n \log n)$ hanya berlaku untuk **comparison sort** — algoritma yang menentukan urutan elemen semata-mata melalui operasi perbandingan ($a < b$, $a > b$, $a == b$). Buktinya menggunakan decision tree: pohon keputusan untuk n elemen memiliki minimal $n!$ daun, sehingga tinggi pohon minimal $\log_2(n!) = \Omega(n \log n)$.

Radix Sort **tidak menggunakan perbandingan antar elemen**. Ia mengeksploitasi struktur internal kunci (digit/bit), sehingga tidak terikat oleh model decision tree. Kedua hasil ini tidak kontradiktif karena berlaku pada model komputasi yang berbeda.

Dua asumsi implisit yang membuat Radix Sort "melampaui" batas comparison sort:

1. **Kunci memiliki panjang digit d yang konstan atau kecil.** Jika $d = O(\log n)$, maka $O(dn) = O(n \log n)$ — sama saja dengan comparison sort. Radix Sort hanya benar-benar linear ($O(n)$) jika d dianggap konstan, yaitu tidak bergantung pada n .
2. **Nilai kunci terbatas dalam rentang k yang terdefinisi.** Setiap digit harus berasal dari alphabet berukuran tetap (misalnya 0–9 atau 0–255). Jika rentang nilai k sangat besar, overhead counting/bucket menjadi $O(k)$ per pass, yang bisa mendominasi $O(n)$.

BAGIAN 2 - Binary Tree & HeapSort

a. Pohon Ekspresi & Traversal

Langkah-langkah `_buildTree()` untuk $((8 * 5) + (9 / (7 - 4)))$:

Token dimasukkan ke deque: (, (, 8, *, 5,), +, (, 9, /, (, 7, -, 4,),),)

Rekursi `_buildTree(tokens)`:

1. Ambil token ($\rightarrow$ buat node internal, rekursi untuk subtree kiri.
2. Rekursi kiri: ambil ($\rightarrow$ rekursi lagi.
 - Rekursi: ambil 8 $\rightarrow$ return node leaf {val:8}.
 - Ambil operator * $\rightarrow$ simpan di node saat ini.
 - Rekursi kanan: ambil 5 $\rightarrow$ return node leaf {val:5}.
 - Ambil) $\rightarrow$ return node {val:'*', left:8, right:5}.
3. Ambil operator + $\rightarrow$ simpan di node root saat ini.
4. Rekursi kanan: ambil ($\rightarrow$ rekursi lagi.
 - Rekursi: ambil 9 $\rightarrow$ leaf.
 - Ambil /.
 - Rekursi kanan: ambil ($\rightarrow$ rekursi lagi.
 - 7 $\rightarrow$ leaf, -, 4 $\rightarrow$ leaf,) $\rightarrow$ node {val:'-', left:7, right:4}.
 -) $\rightarrow$ return node {val:'/', left:9, right:{val:'-', left:7, right:4}}.
5. Ambil) $\rightarrow$ return root {val:'+', left:{val:'*', left:8, right:5}, right:{val:'/', left:9, right:{val:'-', left:7, right:4}}}.

Mengapa postorder menghasilkan postfix otomatis?

Traversal postorder mengunjungi: kiri $\rightarrow$ kanan $\rightarrow$ root. Untuk pohon ekspresi, ini berarti operand kiri diproses dulu, lalu operand kanan, baru operator — persis urutan notasi postfix (RPN). Tidak perlu tanda kurung karena posisi operator sudah implisit dari urutan kunjungan.

Inorder (kiri $\rightarrow$ root $\rightarrow$ kanan) menghasilkan notasi infix, di mana operator muncul di antara operand. Namun infix memerlukan tanda kurung eksplisit untuk menyatakan presedensi, karena urutan operator saja tidak cukup untuk membedakan $(a+b)*c$ dari $a+(b*c)$.

Kedalaman maksimum stack rekursi `_buildString()` untuk pohon tinggi h :

Setiap pemanggilan rekursif turun satu level di pohon. Untuk pohon dengan tinggi h , stack rekursi maksimum = $h + 1$ frame (dari root hingga leaf terdalam). Jadi $O(h)$.

b. Struktur Heap & Pemetaan Array

Mengapa rumus indeks valid hanya untuk complete binary tree?

Rumus $\text{parent} = (i-1)//2$, $\text{left} = 2*i+1$, $\text{right} = 2*i+2$ mengasumsikan bahwa node-node diisi **level by level dari kiri ke kanan tanpa celah**. Ini adalah definisi complete binary tree.

Jika ada "lubang" (node hilang di tengah), pemetaan indeks akan kacau: misalnya, jika node indeks 1 tidak ada, maka indeks 3 dan 4 (yang seharusnya anak dari indeks 1) akan salah dipetakan ke anak dari indeks 2.

Proses `sift_down()` setelah ekstraksi akar:

1. Pindahkan elemen terakhir heap ke posisi akar (indeks 0).
2. Bandingkan akar dengan kedua anaknya ($\text{left} = 1$, $\text{right} = 2$).
3. Jika akar lebih kecil dari anak terbesar, tukar.
4. Ulangi dari posisi baru hingga heap order property terpenuhi ($\text{akar} \geq \text{kedua anak}$) atau mencapai leaf.

Jumlah perbandingan maksimum dalam satu `sift_down()` untuk heap ukuran n :

Setiap level membutuhkan maksimal 2 perbandingan (bandingkan kiri dan kanan, lalu bandingkan parent dengan yang terbesar). Tinggi heap = $\lfloor \log_2 n \rfloor$. Total perbandingan maksimum = $2 \times \lfloor \log_2 n \rfloor = O(\log n)$.

c. **Heapsort In-Place vs Simple**

Aspek	Simple Heapsort	In-Place Heapsort
Ruang tambahan	$O(n)$ — membuat array heap terpisah	$O(1)$ — hanya variabel indeks
Pola akses memori	Lebih cache-friendly saat build heap (array terpisah lebih terlokalisasi)	Kurang cache-friendly karena swap bolak-balik di array yang sama, akses loncat-loncat
Risiko overflow RAM	Tinggi — butuh n elemen tambahan di memori	Sangat rendah — tidak ada alokasi baru

Mengapa in-place heapsort tetap $O(n \log n)$?

- Fase build-heap: $O(n)$ — dibuktikan dengan analisis amortized (sift-down dari bawah ke atas).
- Fase ekstraksi: $n-1$ kali ekstraksi, masing-masing sift-down $O(\log n) \rightarrow$ total $O(n \log n)$.
- Swap hanya $O(1)$ per operasi, tidak mengubah orde kompleksitas.
- Total: $O(n) + O(n \log n) = O(n \log n)$.

d. **Batas Teoretis & Decision Tree**

Mengapa Heapsort tidak melanggar $\Omega(n \log n)$?

Heapsort adalah comparison sort — ia menentukan urutan elemen melalui perbandingan $arr[i] > arr[j]$. Meski memanfaatkan struktur heap (pohon biner), setiap operasi sift-down tetap berbasis perbandingan. Oleh karena itu, Heapsort terikat oleh batas bawah $\Omega(n \log n)$ dan memang mencapai tepat $O(n \log n)$ — bukan melampauinya.

Properti null link untuk mendeteksi token tidak valid pada decision tree (Morse Code):

Pada pohon keputusan Morse Code, setiap node internal memiliki dua anak: kiri (titik $.$) dan kanan (garis $-$). Jika suatu urutan token membawa traversal ke null link

(anak yang tidak ada), berarti urutan tersebut tidak valid dalam alfabet Morse. Ini lebih efisien daripada lookup table karena pengecekan validitas terjadi secara implisit selama traversal.

Mengapa rekursif lebih cocok daripada iteratif untuk pohon keputusan?

Struktur pohon secara inheren rekursif: setiap subtree adalah pohon itu sendiri. Implementasi rekursif memetakan langsung ke definisi struktural pohon — setiap pemanggilan rekursif menangani satu level. Versi iteratif membutuhkan stack eksplisit untuk mensimulasikan rekursi, yang menambah kompleksitas kode tanpa keuntungan algoritmik. Untuk pohon keputusan dengan kedalaman terbatas (Morse Code hanya ~4-5 level), overhead stack rekursi juga sangat kecil.